

Министерство науки и высшего образования Российской Федерации

Курсовая работа

**Автоматическая справочная система для научных материалов на
основе комбинированного retrieval-подхода**

Выполнил: Галимов Тимур

Группа: РМ23-2

Руководитель:

Оглавление

ВВЕДЕНИЕ	4
1. Аналитический раздел	6
1.1. Постановка задачи разработки автоматической справочной системы . . .	6
1.2. Особенности работы с научным корпусом документов	7
1.3. Функциональные требования к системе	11
1.4. Выводы по аналитическому разделу	12
2. Проектирование и реализация программной системы	14
2.1. Архитектурные требования к системе	14
2.2. Общая архитектура системы SciGuide	14
2.3. Архитектура и реализация библиотеки <code>sciguide_pipeline</code>	16
2.4. Реализация прикладного сервиса	17
2.5. Инфраструктура развертывания	18
3. Разработка и исследование retrieval-подхода	19
3.1. Формализация retrieval-задачи для научного корпуса	19
3.2. Сравнительный анализ подходов к поиску по научным документам . . .	19
3.3. Модель выбранного retrieval-пайплайна	20
3.4. Конфигурация и шаги исследуемого пайплайна	21
3.5. Методика эксперимента и метрики оценки	22
3.6. Анализ результатов исследования	23
4. План разработки и оценки обучаемых моделей	25
4.1. Постановка задачи для обучаемого reranking-слоя	25
4.2. Подготовка данных и разбиение выборки	25
4.3. Альтернативные архитектуры моделей	26
4.4. Обучение, функция ошибки и регуляризация	27
4.5. Подбор гиперпараметров и метрики сравнения	27
СПИСОК ЛИТЕРАТУРЫ И ИНТЕРНЕТ-РЕСУРСОВ	28
ПРИЛОЖЕНИЕ А	30
Дополнительные визуальные материалы	30
ПРИЛОЖЕНИЕ Б	32
Ключевые листинги исходного кода	32

ВВЕДЕНИЕ

Рост объема научных материалов усложняет поиск и использование релевантной информации. Обычный поиск по ключевым словам плохо работает в случаях, когда одна и та же идея выражается разными формулировками, а ответы на содержательные вопросы требуют не одного фрагмента, а набора связанных фрагментов из корпуса. При этом универсальные языковые модели без опоры на локальную базу знаний не гарантируют привязку ответа к конкретным документам пользователя¹. Поэтому актуальной является задача разработки автоматической справочной системы, которая отвечает в контексте выбранного научного направления и опирается на загруженный корпус материалов.

Целью курсового проекта является разработка автоматической справочной системы на основе языковых моделей, обеспечивающей ответы по документам выбранного научного направления с использованием комбинированного retrieval-подхода. В качестве основы решения используется двухэтапный поиск: сначала выполняется отбор кандидатов по векторному сходству, затем результаты уточняются с учетом графа сущностей и их связей внутри корпуса.

Для достижения поставленной цели необходимо решить следующие задачи:

1. проанализировать особенности автоматических справочных систем для научных материалов и определить требования к решению;
2. спроектировать архитектуру системы, разделив исследовательский, библиотечный и прикладной уровни;
3. реализовать общее ядро пайплайнов обработки документов и поиска по корпусу в виде отдельной библиотеки;
4. реализовать исследовательский сценарий в Jupyter Notebook для демонстрации и проверки retrieval-пайплайна;
5. разработать прикладной сервис с пользовательскими воркспейсами, документами, чатами и изоляцией контекста;

¹ I. Lewis, P. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks / P. Lewis, E. Perez, A. Piktus [и др.] // Advances in Neural Information Processing Systems. – 2020.. – Т. 33. – Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. – С. 9459-9474

6. описать полученное решение и оценить его применимость для построения научных справочных систем.

Объектом исследования являются автоматические справочные системы, использующие языковые модели для работы с тематическим корпусом документов. Предмет исследования — методы построения retrieval-слоя такой системы, включая семантический поиск, детерминированное извлечение сущностей, графовое представление корпуса и интеграцию этих механизмов в прикладной сервис.

Практическая значимость работы состоит в том, что результатом является не только демонстрационный ноутбук, но и самодостаточное программное решение из трех взаимосвязанных частей. Первая часть представляет собой Jupyter Notebook для исследования и демонстрации retrieval-пайплайна на отдельном датасете. Вторая часть оформлена как библиотека `sciguide_pipeline`, которая инкапсулирует нарезку текста, индексацию корпуса, векторный поиск и графовое перерангирование и может использоваться повторно. Третья часть реализована как сервис SciGuide, предоставляющий пользователю интерфейс работы с воркпейсами, документами и чатами и использующий библиотеку как вычислительное ядро.

Таким образом, в работе рассматривается не только локальный эксперимент с языковой моделью, но и полный путь от retrieval-логики до прикладной системы, пригодной для использования в изолированном пользовательском контексте.

1. Аналитический раздел

1.1. Постановка задачи разработки автоматической справочной системы

Автоматическая справочная система для научных материалов должна решать задачу содержательного ответа на пользовательский вопрос по ограниченному тематическому корпусу документов. В отличие от универсального чат-бота, такая система не должна опираться на произвольные знания модели, а обязана формировать ответ только в рамках загруженных источников. Для курсового проекта это особенно важно, поскольку целевой сценарий предполагает работу с собственным набором документов внутри выбранного научного направления.

Проблема заключается в том, что прямой поиск по ключевым словам плохо переносит вариативность научного языка. Один и тот же смысл может быть выражен разными терминами и синтаксическими конструкциями, а важные для ответа фрагменты нередко связаны не буквальным совпадением слов, а общим контекстом, совместным употреблением понятий и их местом в структуре корпуса. По этой причине пользователю недостаточно просто найти документ; системе требуется отобрать и связать несколько релевантных фрагментов, на основе которых затем формируется ответ.

Из этого следует постановка задачи: необходимо разработать систему, которая принимает корпус документов, индексирует его, находит релевантные фрагменты по пользовательскому запросу и использует найденный контекст для генерации ответа. При этом решение должно обеспечивать изоляцию пользовательских данных, повторное использование retrieval-ядра в разных сценариях и возможность проверки качества на отдельном исследовательском датасете.

В рамках проекта задача решается на трех уровнях. Исследовательский уровень представлен Jupyter Notebook, в котором удобно проводить эксперименты и замеры. Библиотечный уровень оформлен как `sciguide_pipeline` и содержит общее ядро нарезки, индексации и поиска. Прикладной уровень представлен сервисом SciGuide, который обеспечивает работу пользователей с воркспейсами, документами и чатами. Такой разрез позволяет не смешивать экспериментальную логику, переиспользуемое ядро и пользовательский интерфейс в одном монолитном решении.

1.2. Особенности работы с научным корпусом документов

Особенности научного корпуса хорошо видны на примере датасета SciFact, который используется в работе как основной набор данных для проверки retrieval-сценария². Датасет состоит из трех связанных частей: корпуса научных документов, набора коротких утверждений-запросов и разметки релевантности `qrels`, связывающей запрос с документом. В этой постановке целевой переменной является бинарная релевантность документа запросу, а качество поиска далее оценивается по ранжированному списку найденных документов.

В ноутбуке `scifact_dataset_edu.ipynb` были получены базовые характеристики корпуса. Датасет содержит 5183 документа, 1109 запросов и 1258 размеченных пар запрос-документ. Из них 919 пар относятся к обучающей части, а 339 пар — к тестовой; тестовая разметка покрывает 300 запросов и 283 документа. Средняя длина текста документа составляет 201,81 слова, медианная — 192 слова, а средняя длина запроса — 12,38 слова. Для размеченных запросов медианное число релевантных документов равно 1, а максимальное — 5. Сводные показатели корпуса приведены в табл. 1, а визуализация распределений — на рис. 1.

²Wadden, D. Fact or Fiction: Verifying Scientific Claims / D. Wadden, S. Lin, K. Lo [и др.] // Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP). – Association for Computational Linguistics, 2020.. – Fact or Fiction: Verifying Scientific Claims. – С. 7534-7550

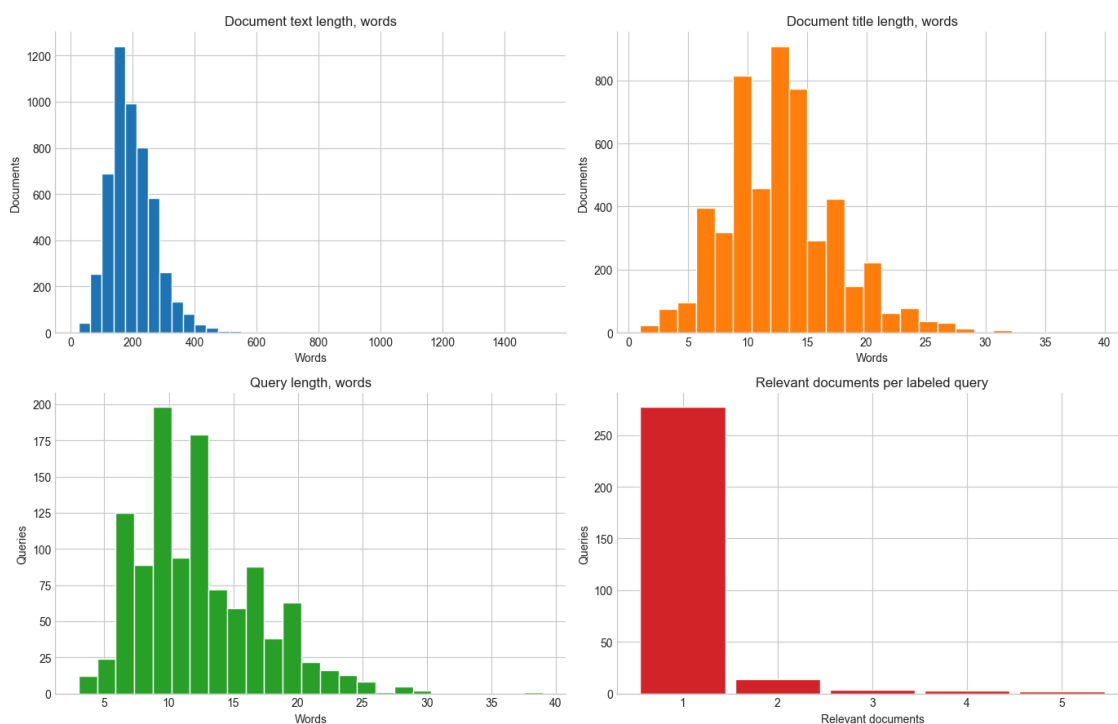


Рисунок 1. Обзор распределений SciFact

Таблица 1. Основные характеристики корпуса SciFact

Показатель	Значение
Количество документов	5183
Количество запросов	1109
Количество размеченных пар запрос-документ	1258
Обучающие размеченные пары	919
Тестовые размеченные пары	339
Тестовые запросы с разметкой	300
Пустые заголовки	0
Пустые тексты документов	0
Средняя длина документа, слов	201,81
Медианная длина документа, слов	192
Средняя длина запроса, слов	12,38
Максимальное число релевантных документов	5

Состав атрибутов датасета приведен в табл. 2. Пропусков, требующих очистки, в ключевых текстовых полях не обнаружено: в корпусе нет пустых заголовков и пустых текстов. Поэтому подготовка данных на этом этапе сводится не к удалению строк, а к приведению документов к единому внутреннему формату: идентификатор документа, заголовок, основной текст и служебные метаданные.

Таблица 2. Атрибуты SciFact и их значение для retrieval-задачи

Объект	Атрибут	Физический смысл и роль в задаче
Документ	<code>_id</code>	Уникальный идентификатор документа; используется для связи с разметкой и результатами поиска.
Документ	<code>title</code>	Заголовок научной публикации; помогает сохранить краткий тематический контекст при индексации.
Документ	<code>text</code>	Основной текст документа; главный источник признаков для векторного поиска, нарезки и извлечения сущностей.
Документ	<code>metadata</code>	Дополнительные сведения о документе; в SciFact в основном пустые, но поле сохраняется для совместимости пайплайна.
Запрос	<code>_id</code>	Уникальный идентификатор запроса; связывает текст запроса с <code>qrels</code> .
Запрос	<code>text</code>	Короткое научное утверждение или вопрос; вход retrieval-модели.
Разметка	<code>query_id, corpus_id</code>	Пара «запрос-документ», для которой задана релевантность.
Разметка	<code>relevance</code>	Целевая переменная: значение 1 означает релевантный документ.
Разметка	<code>split</code>	Обучающая или тестовая часть, добавленная при загрузке файлов <code>qrels</code> .

Структура данных иллюстрируется следующими объектами. Например, документ 4983 имеет заголовок «Microstructural development of human newborn cerebral white matter assessed in vivo by diffusion tensor magnetic resonance imaging.» и содержит 278 слов основного текста. Пример запроса: 0-dimensional biomaterials lack inductive properties. Разметка задает связь вида `query-id = 0, corpus-id = 31715818, score = 1`, то есть документ считается релевантным данному запросу.

Приведенные значения показывают несколько важных свойств научного корпуса.

Во-первых, запросы короткие и информационно сжатые, тогда как документы существенно длиннее и содержат больше терминов, уточнений и контекстных связей. Это означает, что поиск по точным вхождениям слов легко теряет полезные фрагменты, если формулировка в документе не совпадает с формулировкой запроса.

Во-вторых, разметка релевантности разрежена. Для большинства размеченных запросов имеется один релевантный документ, поэтому retrieval-слой должен работать аккуратно уже на ранних этапах отбора кандидатов. Если релевантный фрагмент не попадет в первичную выборку, генерация ответа не сможет опереться на правильный контекст.

В-третьих, документы имеют разную длину, поэтому перед индексацией их необходимо нарезать на сопоставимые фрагменты. В EDA использовалась нарезка с размером окна 900 символов и перекрытием 120 символов. После преобразования 5183 документа дали 15893 фрагмента, в среднем 3,07 фрагмента на документ. Средняя длина фрагмента составила 76,51 слова, медианная — 87 слов. Такая нарезка делает единицей поиска не весь документ, а компактный контекст, который можно передать в генеративную модель.

В-четвертых, научные тексты характеризуются плотностью понятийных связей. При детерминированном извлечении сущностей из фрагментов было найдено 137401 уникальная сущность и 155020 упоминаний; медианное число сущностей на фрагмент равно 12. На основе совместной встречаемости сущностей было построено 257566 графовых связей. Это подтверждает, что релевантность часто определяется не только тем, насколько текст похож на запрос по смыслу, но и тем, какие сущности встречаются рядом, как они связаны между собой и какие локальные кластеры образуют в корпусе. Поэтому одного keyword search недостаточно, а чистый семантический поиск по эмбедингам³, хотя и улучшает

³Reimers, N. Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks / N. Reimers, I. Gurevych // Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP). – Association for Computational Linguistics, 2019.. – Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. – С. 3982-3992

полноту, не всегда различает действительно сильный контекст и просто тематически похожий фрагмент.

По этой причине в работе выбран двухэтапный retrieval-подход, сочетающий retrieval по эмбедингам и структурный сигнал графа⁴. На первом этапе векторный поиск обеспечивает recall и быстро находит кандидатов по семантическому сходству. На втором этапе графовое перерангирование уточняет результат, используя связи между сущностями, извлеченными из корпуса. Такой подход соответствует задаче научной справочной системы: граф не заменяет semantic search, а добавляет structural signal, позволяющий повысить качество итогового контекста для генерации ответа.

1.3. Функциональные требования к системе

Функциональные требования определяют систему не как единичный эксперимент с запросом к модели, а как многопользовательский сервис с изолированными рабочими контекстами.

Базовой сущностью системы является воркспейс. Для каждого воркспейса хранится собственный набор документов и собственный системный промпт, который задает правила поведения модели в рамках данного контекста. Воркспейсы бывают приватными и общими. Приватный воркспейс доступен только владельцу, а общий — назначенным участникам или всем пользователям системы в зависимости от настроек доступа.

Внутри воркспейса различаются две роли: администратор и пользователь. Администратор управляет составом участников, может изменять системный промпт и работать с документами воркспейса. Пользователь взаимодействует с чатами и получает ответы системы, но не изменяет состав корпуса и настройки контекста. Такое разделение необходимо, чтобы разграничить управление базой знаний и ее использование.

Документы воркспейса являются контекстной базой знаний. Система должна обеспечивать их изоляцию между разными воркспейсами, а также запускать обработку и индексацию после загрузки. Это требование напрямую влияет на

⁴1.Lewis, P. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks / P. Lewis, E. Perez, A. Piktus [и др.] // Advances in Neural Information Processing Systems. – 2020.. – Т. 33. – Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. – С. 9459-9474

архитектуру: возникает необходимость в отдельном пайплайне нарезки, извлечения сущностей, построения графа и размещения данных в хранилищах поиска.

Чаты являются личными сущностями пользователя внутри воркспейса. Каждый пользователь может создавать неограниченное число чатов, при этом история сообщений должна сохраняться и быть доступной только владельцу. При генерации ответа система обязана использовать три источника контекста: системный промпт воркспейса, документы текущего воркспейса и историю текущего чата. Использование данных из других воркспейсов или чужих чатов не допускается.

Таким образом, функциональные требования задают сразу несколько обязательных свойств решения: изоляцию данных, разграничение ролей, поддержку фоновой индексации, хранение истории диалога и интеграцию retrieval-слоя в пользовательский сценарий общения с системой.

1.4. Выводы по аналитическому разделу

Проведенный анализ показывает, что разрабатываемая система должна решать две взаимосвязанные, но разные задачи. Первая задача — качественный retrieval по научному корпусу, где важны полнота первичного поиска и учет структурных связей между понятиями. Вторая задача — корректная организация пользовательского сервиса с воркспейсами, ролями, документами, чатами и изоляцией данных.

Из этого следуют архитектурные требования к решению.

1. Retrieval-ядро должно быть выделено в самостоятельный переиспользуемый модуль, чтобы одинаково работать и в исследовательском ноутбуке, и в прикладном сервисе.
2. Индексация документов должна быть отделена от пользовательского запроса, поскольку обработка корпуса требует отдельного конвейера и фоновое выполнение.
3. Хранилища прикладных сущностей и хранилища поиска должны быть разведены по назначению: пользовательские данные целесообразно хранить в транзакционном контуре, а векторные и графовые представления — в специализированных системах.

4. Архитектура backend-части должна сохранять разделение ответственности между слоями предметной области, сценариев использования, инфраструктуры и представления, что соответствует выбранным архитектурным гайдлайнам проекта.

С учетом этих требований обоснованным является следующее решение: построить систему SciGuide как сервис на базе clean architecture, выделить библиотеку `sciguide_pipeline` в качестве общего вычислительного ядра и использовать двух-этапный retrieval, объединяющий векторный поиск и графовое перерангирование. Такой подход позволяет совместить исследовательскую воспроизводимость, переиспользуемость кода и прикладную пригодность решения.

2. Проектирование и реализация программной системы

2.1. Архитектурные требования к системе

Требования к программной системе напрямую следуют из постановки задачи и функциональных требований. Во-первых, система должна изолировать данные разных воркспейсов: документы, системный промпт, чаты и история сообщений не должны смешиваться между пользователями и рабочими контекстами. Во-вторых, требуется разделить прикладные данные и данные retrieval-контура. Пользовательские сущности удобно хранить в транзакционном хранилище, тогда как векторные представления и граф связей должны размещаться в специализированных системах.

Отдельным требованием является фоновая индексация документов. Загрузка файла не должна блокировать пользовательский интерфейс, поэтому обработка корпуса вынесена в асинхронный контур. Кроме того, retrieval-ядро должно использоваться в двух режимах: в исследовательском ноутбуке и в прикладном сервисе. Это привело к выделению общей библиотеки `sciguide_pipeline`.

Архитектурные гайдлайны проекта задают использование clean architecture в backend-части⁵. Это означает разделение на доменный слой, слой сценариев использования, инфраструктурный слой и слой представления. Такое разбиение уменьшает связность между бизнес-логикой и внешними технологиями и упрощает интеграцию retrieval-библиотеки в разные сценарии.

2.2. Общая архитектура системы SciGuide

На верхнем уровне система состоит из web-клиента, backend-сервиса, фонового worker-процесса, общей библиотеки пайплайнов и набора специализированных хранилищ. Общая структура показана на рис. 2.

⁵4.Martin, R. C. Clean Architecture: A Craftsman's Guide to Software Structure and Design. Clean Architecture: A Craftsman's Guide to Software Structure and Design / R. C. Martin. – Prentice Hall, 2018.

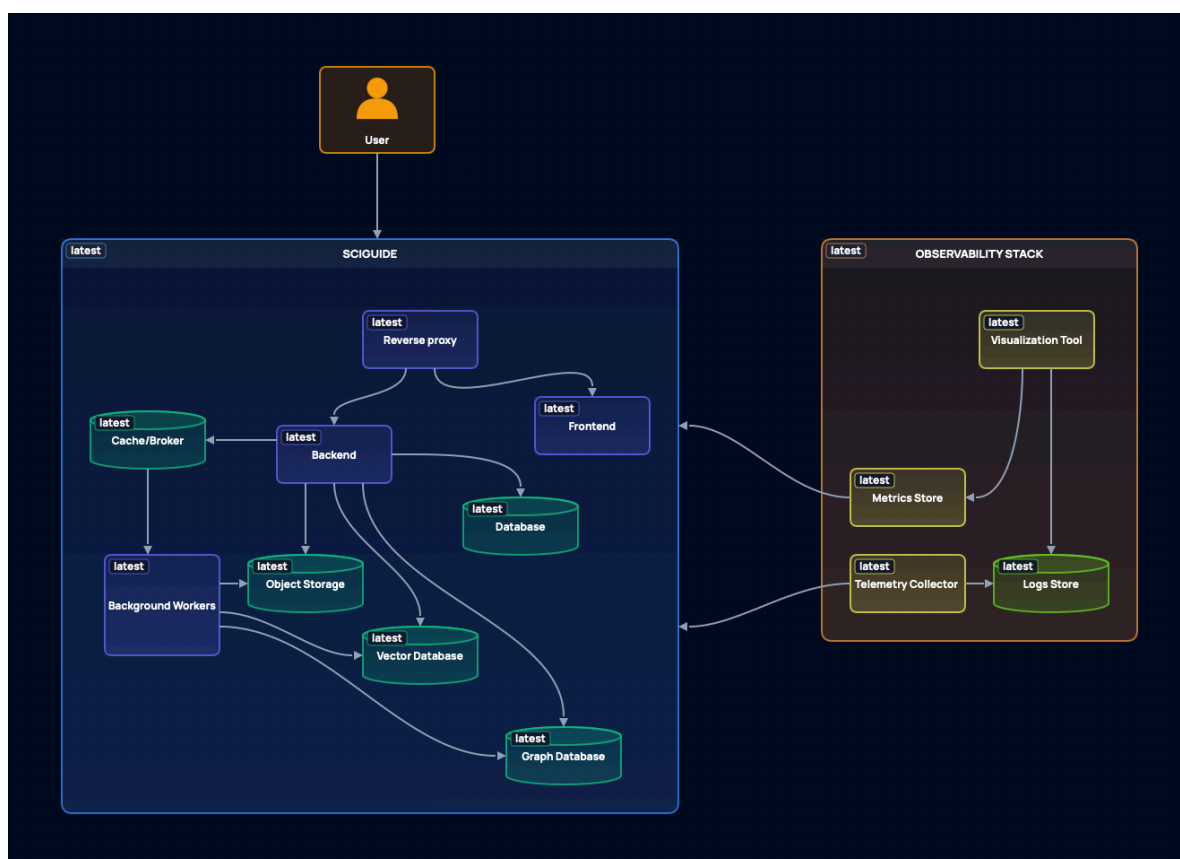


Рисунок 2. C4-диаграмма системы SciGuide

Frontend реализует пользовательский интерфейс для работы с воркспейсами, документами и чатами. Backend на FastAPI предоставляет HTTP API, управляет прикладными сущностями и координирует сценарии загрузки документов и генерации ответов. Celery-worker выполняет фоновую индексацию документов. Библиотека `sciguide_pipeline` инкапсулирует retrieval-логику и используется как worker-ом, так и backend-частью при поиске контекста для ответа.

Хранилища распределены по функциям. PostgreSQL используется для пользователей, воркспейсов, чатов и сообщений. Redis служит брокером задач и кэшем. MinIO хранит загруженные файлы. Qdrant хранит векторные представления фрагментов текста⁶. Neo4j содержит граф сущностей и их связей⁷. На входе в систему размещен Nginx, который выступает единой точкой доступа.

Поток обработки имеет следующий вид: документ загружается в воркспейс и сохраняется в MinIO; backend ставит задачу на индексацию; worker запускает библиотеку `sciguide_pipeline`; результаты нарезки и эмбединги сохраняются

⁶5.Qdrant Team. Qdrant Documentation. – URL: <https://qdrant.tech/documentation/> (дата обращения: 20.04.2026). – Текст : электронный

⁷6.Neo4j, Inc. Neo4j Documentation. – URL: <https://neo4j.com/docs/> (дата обращения: 20.04.2026). – Текст : электронный

в Qdrant, а граф сущностей — в Neo4j; при отправке сообщения backend выполняет retrieval по текущему воркспейсу и передает найденный контекст языковой модели.

2.3. Архитектура и реализация библиотеки `sciguide_pipeline`

Библиотека `sciguide_pipeline` выделена в отдельный технический компонент, чтобы не дублировать retrieval-логику в ноутбуке и в сервисе. Пользователь библиотеки работает с одной точкой входа — `PipelineManager`, который предоставляет доступ к двум фасадам: `chunking` и `search`. Минимальный пример использования фасада показан в листинг 1.

```
with PipelineManager(...) as manager:
    manager.chunking.run([source_document])
    report = manager.search.run(query="Что такое graph-guided
    retrieval?")
```

Листинг 1. Минимальный сценарий использования `PipelineManager`

Внутри `PipelineManager` создаются и связываются основные компоненты пайплайна: `LangChainTextChunker`, `DeterministicEntityExtractor`, сервис эмбедингов, репозиторий векторного поиска `QdrantVectorRepository`, графовый репозиторий `Neo4jGraphRepository` и `WeightedScoreCombiner`. Таким образом, библиотека скрывает технические детали подключения к внешним системам, управляет клиентами, чтобы не перегружать исходящий трафик и предоставляет единый пользовательский контракт.

С технической точки зрения библиотека организована как последовательность согласованных шагов обработки, скрытых за единым API. Она задает собственные контракты для входных данных, результатов пайплайнов и точек расширения. Благодаря этому и в ноутбуке, и в backend-сервисе на вход передаются одинаковые сущности — содержимое документа и связанные метаданные, — а не объекты, специфичные для конкретного интерфейса загрузки или web-фреймворка. Такое решение делает библиотеку самостоятельным техническим модулем, который можно использовать независимо от прикладной оболочки.

2.4. Реализация прикладного сервиса

Backend реализован в соответствии с принципами clean architecture и разделен на доменные области: auth, workspaces, workspace_documents, workspace_prompt, chats, messages, workspace_members. Каждая область содержит собственные слои domain, application, infrastructure и presentation. Компоновка HTTP API выполняется на уровне общего роутера, показанного в листинг 2:

```
api_router = APIRouter(prefix="/api/v1")
api_router.include_router(auth_router)
api_router.include_router(workspaces_router)
api_router.include_router(documents_router)
api_router.include_router(chats_router)
api_router.include_router(messages_router)
```

Листинг 2. Подключение HTTP-маршрутов на уровне общего API-роутера

Индексация документа реализована как отдельный сценарий использования. После загрузки файл читается из хранилища, для него обновляется статус обработки, затем вызывается индексатор на основе PipelineManager, а по завершении документ переводится в состояние PROCESSED. Ключевой вызов индексатора приведен в листинг 3.

```
await self._document_indexer.index(
    document=document,
    content_bytes=content_bytes,
)
```

Листинг 3. Вызов индексатора документа в сценарии обработки

Генерация ответа строится аналогично: сервис сначала извлекает контекст текущего воркспейса через sciguide_pipeline, а затем передает найденные фрагменты языковой модели. В OpenRouterAssistantResponder retrieval выполняется для коллекции, связанной с конкретным workspace_id, что обеспечивает изоляцию поискового контекста между воркспейсами.

Frontend реализован на Vue и выступает пользовательской оболочкой над backend API. Его задача — предоставить сценарии регистрации, работы с воркспейсами, загрузки документов, просмотра истории обработки и общения с системой в чате.

2.5. Инфраструктура развертывания

Система рассчитана на контейнерное развертывание. Основные сервисы поднимаются через Docker Compose, что позволяет воспроизвести рабочее окружение целиком: backend, frontend, worker, базы данных и retrieval-хранилища. На входе используется Nginx, а фоновая индексация реализована через Celery с Redis в роли брокера и backend-хранилища задач. Базовая конфигурация Celery приведена в листинг 4.

```
celery_app = Celery(  
    "sciguide",  
    broker=_build_redis_url(),  
    backend=_build_redis_url(),  
    include=["workspace_documents.infrastructure.tasks"],  
)
```

Листинг 4. Конфигурация Celery для фоновой индексации документов

Backend поднимает подключения к PostgreSQL и Redis в lifespan-контексте FastAPI, а также публикует healthcheck и HTTP-метрики. Для сопровождения системы предусмотрены Prometheus, Grafana, Loki, Grafana Alloy и формат телеметрии OpenTelemetry.

В результате реализованная инфраструктура поддерживает как исследовательский режим, так и эксплуатационный сценарий: пользователь может загрузить документ, дождаться фоновой индексации и получить ответ, сформированный на основе контекста текущего воркспейса.

3. Разработка и исследование retrieval-подхода

3.1. Формализация retrieval-задачи для научного корпуса

Retrieval-задача в работе рассматривается как задача ранжирования документов научного корпуса по пользовательскому запросу. На вход подается запрос (q) и набор документов ($D = \{d_1, d_2, \dots, d_n\}$). Каждый документ предварительно разбивается на фрагменты, поскольку в сценарии справочной системы языковой модели передается не весь корпус, а ограниченный контекст. Retrieval-пайплайн должен вернуть упорядоченный список документов или фрагментов, где релевантные элементы находятся как можно выше.

Для датасета SciFact релевантность задана в `qrels`: пара `query_id` и `corpus_id` имеет оценку 1, если документ подтверждает или опровергает соответствующее научное утверждение. Поэтому базовая проверка качества выполняется на уровне документов: фрагменты используются как единицы поиска, но их оценки агрегируются к идентификаторам исходных документов. Такой подход соответствует прикладному сценарию SciGuide: система сначала находит компактные фрагменты, но пользовательский ответ должен быть привязан к исходным источникам.

Достаточным результатом retrieval-пайплайна считается такой список кандидатов, в котором релевантный документ попадает в верхние позиции выдачи. Для генерации ответа особенно важны значения на малых k , поскольку в контекст языковой модели нельзя передать большое число фрагментов. Поэтому в качестве основных ориентиров используются метрики `nDCG@5`, `nDCG@10`, `Recall@5`, `Recall@10` и `MRR@10`.

3.2. Сравнительный анализ подходов к поиску по научным документам

В ноутбуке `scifact_retrieval_baselines.ipynb` сравниваются три базовых подхода к retrieval. Они не являются обучаемыми моделями глубокого обучения; их роль — задать исходный уровень качества, с которым далее можно сравнивать более сложные архитектуры.

Первый подход — `BM25 / keyword baseline`. Он использует лексическое совпадение между токенами запроса и фрагментов документа. Его преимущество

состоит в простоте, воспроизводимости и хорошей работе по точным терминам. Недостаток проявляется в научном корпусе: если один и тот же смысл выражен разными словами, keyword-поиск может не найти релевантный документ.

Второй подход — `Semantic search`. Для каждого фрагмента и запроса строится `embedding`-представление, после чего используется косинусное сходство. Такой подход лучше переносит различия в формулировках и ближе к задаче поиска по смыслу⁸. При этом он может поднимать в выдаче тематически похожие, но не обязательно релевантные фрагменты.

Третий подход — `Semantic + graph rerank`. Он сохраняет семантический поиск как основной источник кандидатов, но добавляет структурный сигнал: из фрагментов извлекаются сущности, строятся связи совместной встречаемости, а итоговый скоринг учитывает не только векторное сходство, но и совпадение сущностей запроса с локальным графовым контекстом. Этот вариант ближе к целевой архитектуре SciGuide, где граф не заменяет `embeddings`, а уточняет ранжирование.

Отдельно можно рассмотреть прямое обращение к LLM без `retrieval`, но для данной задачи это не является корректным `baseline`. Такая схема не гарантирует привязку ответа к документам пользователя и не позволяет измерить качество поиска по `qrels`. Поэтому в экспериментальной части она не сравнивается как `retrieval`-пайплайн, а рассматривается как неподходящая альтернатива для справочной системы с проверяемыми источниками⁹.

3.3. Модель выбранного `retrieval`-пайплайна

Общая схема исследуемого пайплайна включает пять шагов.

1. Документы SciFact приводятся к единому формату: заголовок и текст объединяются в содержимое документа, а исходный идентификатор сохраняется в метаданных.

⁸3.Reimers, N. Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks / N. Reimers, I. Gurevych // Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP). – Association for Computational Linguistics, 2019.. – Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. – С. 3982-3992

⁹1.Lewis, P. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks / P. Lewis, E. Perez, A. Piktus [и др.] // Advances in Neural Information Processing Systems. – 2020.. – Т. 33. – Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. – С. 9459-9474

2. Документы разбиваются на фрагменты фиксированного размера. Фрагмент становится минимальной единицей поиска, а документ — единицей итоговой оценки.
3. Для keyword baseline выполняется токенизация и строится BM25-индекс по фрагментам.
4. Для semantic baseline строятся embedding-векторы фрагментов и запросов; сходство вычисляется через скалярное произведение нормализованных векторов.
5. Для графового варианта из фрагментов извлекаются сущности, строится граф совместной встречаемости, после чего векторный скор объединяется со структурным скором. Вес структурного сигнала не задается вручную, а подбирается на валидационной части обучающей разметки.

Итоговая оценка документа получается агрегацией оценок его фрагментов: среди верхних найденных фрагментов для каждого документа сохраняется максимальный скор. Это важно для научных текстов, потому что релевантным может быть не весь документ целиком, а отдельный абзац или фрагмент с нужным утверждением.

3.4. Конфигурация и шаги исследуемого пайплайна

Для baseline-эксперимента использовался полный корпус SciFact: 5183 документа, 300 тестовых запросов, 339 тестовых размеченных пар и 283 релевантных документа в тестовой разметке. Отдельно из обучающей части `qrels/train.tsv` была выделена валидационная часть из 161 запроса. Она использовалась только для выбора веса графового сигнала, тогда как финальные метрики считались на тестовой разметке.

Основные параметры эксперимента приведены в табл. 3.

Таблица 3. Конфигурация baseline-эксперимента

Параметр	Значение
Количество документов в корпусе	5183
Количество тестовых запросов	300
Количество тестовых размеченных пар	339
Количество релевантных документов в test	283
Количество validation-запросов	161
Размер фрагмента	900 символов
Перекрытие фрагментов	120 символов
Количество фрагментов после нарезки	15893
Среднее число токенов во фрагменте	54,58
Среднее число токенов в тестовом запросе	9,42
Embedding-модель	BAAI/bge-m3
Размерность embedding-вектора	1024
Сетка веса структурного сигнала	0,00; 0,05; 0,10; 0,15; 0,20; 0,25
Выбранный вес структурного сигнала	0,05

В графовом варианте было извлечено 137401 сущность и 257566 связей, среднее число сущностей на фрагмент составило 9,75. Чтобы графовый сигнал не ухудшал ранжирование за счет слишком общих терминов, сущности взвешивались по IDF, а вклад соседних сущностей был ослаблен коэффициентом 0,15. Итоговый структурный вес подбирался на validation-разметке; лучшим оказался вес 0,05. Тем самым semantic search остается основой ранжирования, а графовый сигнал используется как слабая корректировка, а не как равноправная замена векторного сходства.

3.5. Методика эксперимента и метрики оценки

Все три baseline-подхода оценивались на одном и том же полном тестовом наборе SciFact, с одинаковой нарезкой документов и одинаковой агрегацией фрагментов к документам. Это позволяет сравнивать именно способ скоринга, а не различия в подготовке данных. Подбор веса графового сигнала выполнялся до финальной оценки и не использовал тестовую разметку.

Для каждого запроса пайплайн возвращал ранжированный список документов. Далее этот список сопоставлялся с `qrels`, а качество измерялось по метрикам, приведенным в табл. 4.

Таблица 4. Метрики оценки retrieval-качества

Метрика	Что показывает
nDCG@k	Насколько высоко в выдаче расположены релевантные документы с учетом позиции.
Recall@k	Какая доля релевантных документов попала в первые k результатов.
Precision@k	Какая доля первых k результатов является релевантной.
MRR@k	Насколько рано появляется первый релевантный документ.

Для справочной системы наиболее важны Recall@k и MRR@k. Высокий Recall@k означает, что нужный источник вообще попал в контекст, а высокий MRR@k показывает, что он находится близко к началу выдачи. nDCG@k используется как более общий показатель качества ранжирования, а Precision@k помогает контролировать долю лишнего контекста.

3.6. Анализ результатов исследования

Результаты baseline-эксперимента приведены в табл. 5.

Таблица 5. Сравнение baseline retrieval-пайплайнов

Пайплайн	nDCG@5	nDCG@10	Recall@5	Recall@10	Precision@5	Precision@10	MRR@5	MRR@10
BM25 / keyword	0,5928	0,6100	0,6886	0,7367	0,1467	0,0803	0,5681	0,5736
Semantic search	0,6214	0,6488	0,7025	0,7811	0,1547	0,0877	0,6056	0,6150
Semantic + graph rerank (w = 0,05)	0,6281	0,6537	0,7242	0,7968	0,1593	0,0897	0,6065	0,6151

Keyword baseline оказался достаточно сильным: для научных утверждений точные термины часто имеют большое значение, поэтому BM25 находит часть релевантных документов. Однако semantic search заметно улучшил все ключевые метрики: nDCG@10 вырос с 0,6100 до 0,6488, а Recall@10 — с 0,7367 до 0,7811. Это подтверждает, что embedding-поиск лучше расширяет первичную выдачу и чаще включает релевантный документ в верхние 10 результатов.

Графовое переранжирование после подбора веса дало небольшой, но устойчивый прирост относительно чистого semantic search. Значение $nDCG@10$ выросло с 0,6488 до 0,6537, $Recall@10$ — с 0,7811 до 0,7968, $Precision@10$ — с 0,0877 до 0,0897. $MRR@10$ практически не изменился: 0,6150 против 0,6151. Это означает, что графовый сигнал в текущем виде не радикально меняет верхнюю позицию выдачи, но помогает включать больше релевантных документов в top-10 и немного улучшает общий порядок ранжирования.

Из результатов следует практический вывод: для SciGuide целесообразно использовать semantic retrieval как основной механизм отбора кандидатов, а графовый сигнал — как слабый дополнительный слой ранжирования с весом, выбранным на validation-разметке. Такой baseline не решает задачу обучения модели, но задает измеримую точку отсчета. Следующий этап работы с моделями глубокого обучения должен сравниваться не с пустым решением, а с уже работающим retrieval-пайплайном, который обеспечивает сильный и интерпретируемый базовый уровень качества.

4. План разработки и оценки обучаемых моделей

4.1. Постановка задачи для обучаемого reranking-слоя

Следующий этап исследования связан с переходом от эвристического графового переранжирования к обучаемому слою ранжирования. Базовый retrieval-пайплайн из раздела 3 уже формирует список кандидатов, поэтому обучаемая модель должна не заменять весь поиск, а уточнять порядок найденных документов и фрагментов.

Задача формулируется как предсказание релевантности пары «запрос-документ» или «запрос-фрагмент». Положительные пары берутся из `qrels`, а отрицательные формируются с помощью `negative sampling`. Для повышения практической сложности эксперимента целесообразно использовать не только случайные отрицательные документы, но и `hard negatives` из верхних результатов `baseline`-поиска: такие документы похожи на запрос, но не размечены как релевантные.

Входными признаками модели должны быть текстовые `embedding`-представления запросов, документов и фрагментов, а также графовые связи между сущностями, фрагментами и документами. На выходе модель возвращает `score` релевантности, который используется для повторного ранжирования кандидатов.

4.2. Подготовка данных и разбиение выборки

Для обучения предполагается использовать обучающую часть `qrels/train.tsv`, а тестовую часть `qrels/test.tsv` оставить только для финальной оценки. Из обучающей части необходимо выделить валидационную выборку, например стратифицированным разделением по запросам. Такое разделение снижает риск утечки, при которой один и тот же запрос одновременно участвует в подборе параметров и финальной проверке.

Граф для обучения строится как гетерогенная структура. Узлами являются запросы, документы, фрагменты и извлеченные сущности. Ребра отражают связи `document-chunk`, `chunk-entity`, `entity-entity` и размеченные пары `query-document`. Такой формат соответствует уже реализованному retrieval-пайплайну

и позволяет использовать специализированные библиотеки для графового обучения, например PyTorch Geometric¹⁰.

4.3. Альтернативные архитектуры моделей

В качестве трех основных архитектур предлагается сравнить GraphSAGE, GAT и R-GCN.

GraphSAGE используется как базовая графовая нейросетевая архитектура. Она обучает функцию агрегации соседей, а не отдельный embedding для каждого узла, поэтому подходит для сценария, где в систему могут добавляться новые документы и фрагменты¹¹.

GAT расширяет эту идею механизмом attention: модель может назначать разные веса соседям узла и тем самым учитывать, что не все сущности и связи одинаково полезны для определения релевантности¹². Для SciGuide это важно, поскольку часть сущностей является общей и слабо информативной, а часть прямо указывает на научный контекст запроса.

R-GCN рассматривается как наиболее естественный вариант для гетерогенного графа, поскольку он учитывает типы отношений между узлами¹³. В этой работе типы связей имеют самостоятельный смысл: документ состоит из фрагментов, фрагмент содержит сущности, сущности связаны совместной встречаемостью, а запрос связан с релевантным документом.

Как внешние ориентиры для neural retrieval можно также рассматривать dual-encoder подходы, например DPR¹⁴, и late interaction архитектуры, например

¹⁰7. PyTorch Geometric Team. PyTorch Geometric Documentation. – URL: <https://pytorch-geometric.readthedocs.io/> (дата обращения: 26.04.2026). – Текст : электронный

¹¹8. Hamilton, W. L. Inductive Representation Learning on Large Graphs / W. L. Hamilton, R. Ying, J. Leskovec // Advances in Neural Information Processing Systems. – 2017.. – Т. 30. – Inductive Representation Learning on Large Graphs. – С. 1024-1034

¹²9. Veličković, P. Graph Attention Networks / P. Veličković, G. Cucurull, A. Casanova [и др.] // International Conference on Learning Representations. – 2018.. – Graph Attention Networks

¹³10. Schlichtkrull, M. Modeling Relational Data with Graph Convolutional Networks / M. Schlichtkrull, T. N. Kipf, P. Bloem [и др.] // The Semantic Web: 15th International Conference, ESWC 2018 : Lecture Notes in Computer Science. – Springer, 2018.. – Т. 10843. – Modeling Relational Data with Graph Convolutional Networks. – С. 593-607

¹⁴11. Karpukhin, V. Dense Passage Retrieval for Open-Domain Question Answering / V. Karpukhin, B. Oguz, S. Min [и др.] // Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP). – Association for Computational Linguistics, 2020.. – Dense Passage Retrieval for Open-Domain Question Answering. – С. 6769-6781

ColBERT¹⁵. Однако в рамках данной курсовой основной акцент целесообразно оставить на графовых моделях, потому что baseline уже показал пользу структурного сигнала.

4.4. Обучение, функция ошибки и регуляризация

Основной функцией ошибки для первого этапа можно выбрать BCEWithLogitsLoss, где положительные пары соответствуют релевантным документам, а отрицательные — нерелевантным кандидатам. Дополнительно может быть проверена pairwise ranking loss, в которой модель учится присваивать положительному документу больший score, чем отрицательному для того же запроса.

Для контроля переобучения необходимо использовать dropout, weight decay и early stopping по валидационной метрике. Также следует ограничить глубину графовой модели двумя или тремя слоями: более глубокие GNN могут сильнее смешивать локальные представления и хуже работать на разреженной разметке.

4.5. Подбор гиперпараметров и метрики сравнения

Подбор гиперпараметров должен включать не менее трех параметров: learning rate, размер скрытого слоя, dropout, число GNN-слоев и долю отрицательных примеров. Для одного из параметров следует выполнить автоматический перебор, например grid search по learning rate или dropout.

Сравнение моделей должно проводиться теми же retrieval-метриками, что и baseline: nDCG@5, nDCG@10, Recall@5, Recall@10, MRR@10 и Precision@10. Это позволит напрямую сопоставить обучаемые модели с BM25, semantic search и semantic + graph rerank. Дополнительно необходимо фиксировать train loss, validation loss, время обучения и разрыв между train- и validation-метриками, чтобы проверить модель на переобучение.

Итоговая таблица четвертого раздела должна включать baseline из раздела 3 и три обучаемые архитектуры. Лучшей следует считать не только модель с максимальной метрикой, но и модель с устойчивым качеством на валидации и тесте, разумным временем обучения и понятной применимостью в сервисе SciGuide.

¹⁵12.Khattab, O. ColBERT: Efficient and Effective Passage Search via Contextualized Late Interaction over BERT / O. Khattab, M. Zaharia // Proceedings of the 43rd International ACM SIGIR Conference on Research and Development in Information Retrieval. — Association for Computing Machinery, 2020.. — ColBERT: Efficient and Effective Passage Search via Contextualized Late Interaction over BERT. — С. 39-48

СПИСОК ЛИТЕРАТУРЫ И ИНТЕРНЕТ-РЕСУРСОВ

1. Lewis, P. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks / P. Lewis, E. Perez, A. Piktus [и др.] // Advances in Neural Information Processing Systems. – 2020.. – Т. 33. – Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. – С. 9459-9474.
2. Wadden, D. Fact or Fiction: Verifying Scientific Claims / D. Wadden, S. Lin, K. Lo [и др.] // Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP). – Association for Computational Linguistics, 2020.. – Fact or Fiction: Verifying Scientific Claims. – С. 7534-7550.
3. Reimers, N. Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks / N. Reimers, I. Gurevych // Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP). – Association for Computational Linguistics, 2019.. – Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. – С. 3982-3992.
4. Martin, R. C. Clean Architecture: A Craftsman's Guide to Software Structure and Design. Clean Architecture: A Craftsman's Guide to Software Structure and Design / R. C. Martin. – Prentice Hall, 2018.
5. Qdrant Team. Qdrant Documentation. – URL: <https://qdrant.tech/documentation/> (дата обращения: 20.04.2026). – Текст : электронный.
6. Neo4j, Inc. Neo4j Documentation. – URL: <https://neo4j.com/docs/> (дата обращения: 20.04.2026). – Текст : электронный.
7. PyTorch Geometric Team. PyTorch Geometric Documentation. – URL: <https://pytorch-geometric.readthedocs.io/> (дата обращения: 26.04.2026). – Текст : электронный.
8. Hamilton, W. L. Inductive Representation Learning on Large Graphs / W. L. Hamilton, R. Ying, J. Leskovec // Advances in Neural Information Processing Systems. – 2017.. – Т. 30. – Inductive Representation Learning on Large Graphs. – С. 1024-1034.
9. Veličković, P. Graph Attention Networks / P. Veličković, G. Cucurull, A. Casanova [и др.] // International Conference on Learning Representations. – 2018.. – Graph Attention Networks.

- 10.Schlichtkrull, M. Modeling Relational Data with Graph Convolutional Networks / M. Schlichtkrull, T. N. Kipf, P. Bloem [и др.] // The Semantic Web: 15th International Conference, ESWC 2018 : Lecture Notes in Computer Science. – Springer, 2018.. – T. 10843. – Modeling Relational Data with Graph Convolutional Networks. – C. 593-607.
- 11.Karpukhin, V. Dense Passage Retrieval for Open-Domain Question Answering / V. Karpukhin, B. Oguz, S. Min [и др.] // Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP). – Association for Computational Linguistics, 2020.. – Dense Passage Retrieval for Open-Domain Question Answering. – C. 6769-6781.
- 12.Khattab, O. ColBERT: Efficient and Effective Passage Search via Contextualized Late Interaction over BERT / O. Khattab, M. Zaharia // Proceedings of the 43rd International ACM SIGIR Conference on Research and Development in Information Retrieval. – Association for Computing Machinery, 2020.. – ColBERT: Efficient and Effective Passage Search via Contextualized Late Interaction over BERT. – C. 39-48.

ПРИЛОЖЕНИЕ А

Дополнительные визуальные материалы

В приложении собраны укрупненные иллюстрации, на которые опирается аналитическая и проектная часть работы.

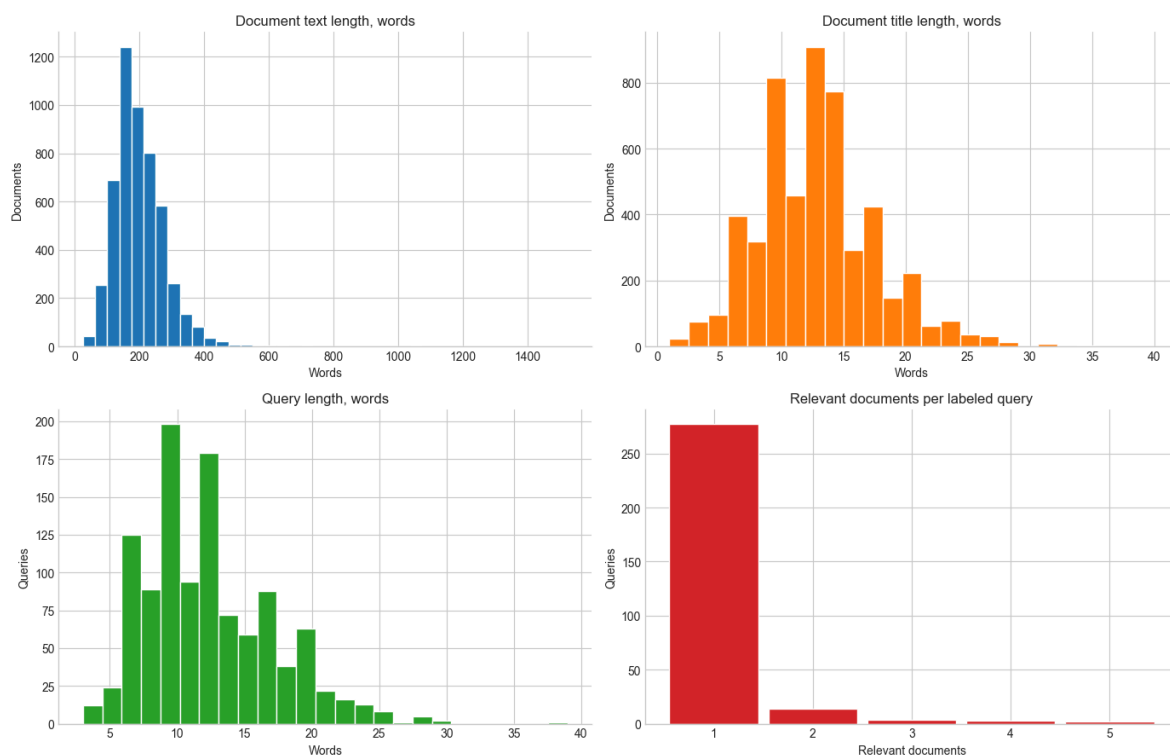


Рисунок 3. Распределения основных характеристик корпуса SciFact

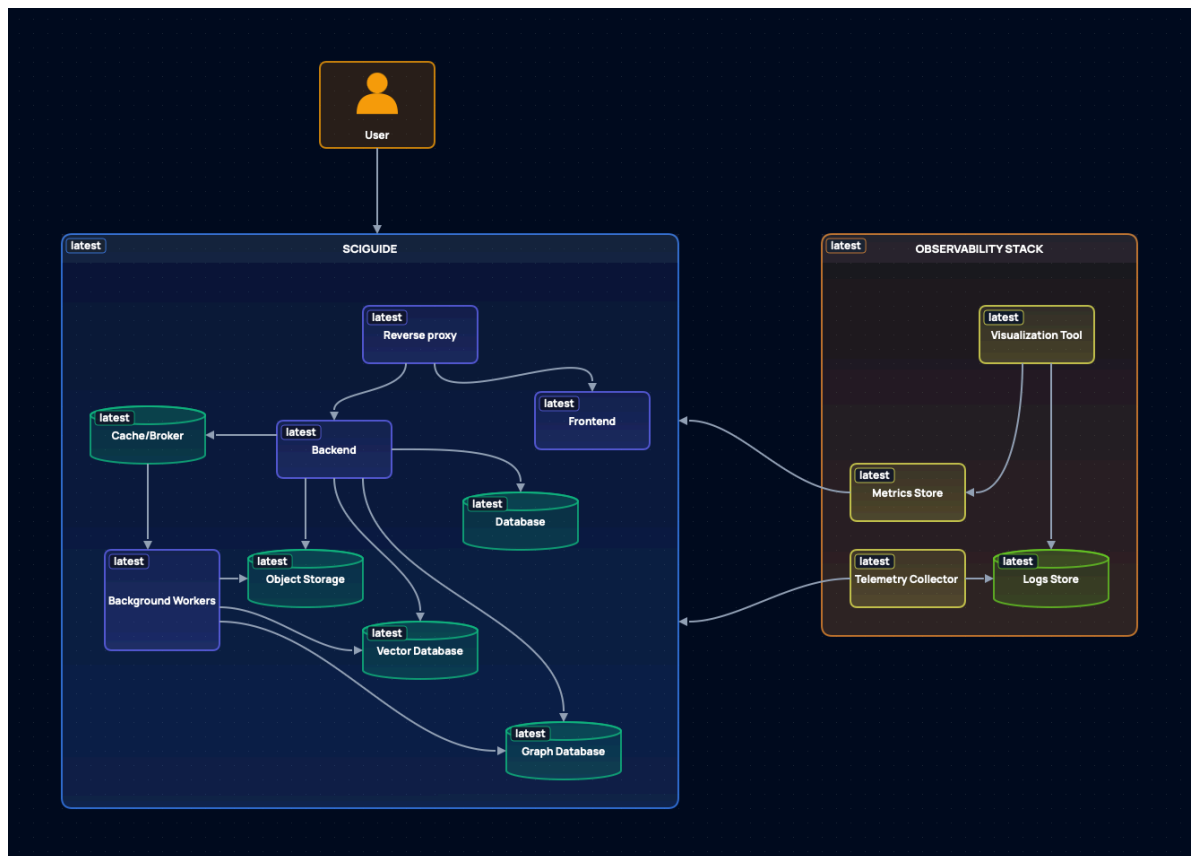


Рисунок 4. Архитектурная схема системы SciGuide

ПРИЛОЖЕНИЕ Б

Ключевые листинги исходного кода

В приложении приведены расширенные фрагменты исходного кода, которые показывают организацию retrieval-контура и фоновую обработку документов.

```
class RunChunking:
    """Orchestrates chunking, embedding, and persistence."""

    def execute(self, request: RunChunkingRequest) -> ChunkingReport:
        """Execute the end-to-end chunking workflow."""
        chunks =
self._text_chunker.chunk_documents(request.documents)
        if not chunks:
            return ChunkingReport(
                documents_processed=len(request.documents),
                chunks_created=0,
                collection_name=self._collection_name,
                graph_namespace=self._graph_namespace,
            )

        enriched_chunks = [
            replace(
                chunk,
                entities=tuple(self._entity_extractor.extract(chunk.text)),
            )
            for chunk in chunks
        ]

        embeddings = self._embedding_service.embed_documents(
            [chunk.text for chunk in enriched_chunks]
        )

        self._vector_repository.ensure_collection(
            self._embedding_service.vector_size
        )
        self._vector_repository.upsert_chunks(enriched_chunks,
embeddings)
        self._graph_repository.ensure_schema()
        self._graph_repository.upsert_chunks(enriched_chunks)
```

Листинг 5. Сценарий RunChunking: оркестрация chunking, embedding и сохранения


```

class PipelineManager:
    """Main developer-facing entrypoint for the SciGuide library."""

    def __init__(
        self,
        qdrant_url: str,
        qdrant_collection_name: str,
        neo4j_uri: str,
        neo4j_username: str,
        neo4j_password: str,
        embedding_model_name: str,
        reranker_model_name: str,
        model_cache_dir: str | Path,
        *,
        graph_namespace: str | None = None,
        qdrant_api_key: str | None = None,
        qdrant_prefer_grpc: bool = False,
        neo4j_database: str = "neo4j",
        huggingface_token: str | None = None,
        chunk_size: int = 1200,
        chunk_overlap: int = 200,
        search_limit: int = 5,
        search_candidate_limit: int = 20,
        vector_weight: float = 0.45,
        graph_weight: float = 0.20,
        rerank_weight: float = 0.35,
        request_timeout: float = 60.0,
    ) -> None:
        ...
        self.chunking = ChunkingPipeline(self._chunking_use_case)
        self.search = SearchPipeline(
            self._search_use_case,
            default_limit=search_limit,
            default_candidate_limit=search_candidate_limit,
        )

```

Листинг 6. Класс PipelineManager: сборка фасадов chunking и search

```
celery_app = Celery(
    "sciguide",
    broker=_build_redis_url(),
    backend=_build_redis_url(),
    include=["workspace_documents.infrastructure.tasks"],
)

celery_app.conf.update(
    task_serializer="json",
    accept_content=["json"],
    result_serializer="json",
    task_default_queue="workspace-document-indexing",
    task_track_started=True,
    timezone="UTC",
    enable_utc=True,
)
```

Листинг 7. Конфигурация Celery для фоновой индексации документов